

TPFI 2025/26

Hw 4: Effectful Haskell

assegnato: 13 maggio 2026, consegna 26 maggio 2026

1. Input/Output

Definite un'azione `adder :: IO ()` che legge un numero n da tastiera, poi n numeri interi e alla fine stampa la loro somma.

2. Applicativi

Un numero n è *perfetto* se è uguale alla somma dei suoi divisori propri (compreso l'1 ed escluso n). Ad esempio, il $28 = 1 + 2 + 4 + 7 + 14$. Tutti gli altri numeri sono classificabili come *difettivi* (se la somma dei divisori è minore del numero, ad esempio tutti i primi sono difettivi) oppure *abbondanti* (se la somma dei divisori è maggiore del numero, ad esempio il $24 < 1 + 2 + 3 + 4 + 6 + 8 + 12 = 36$).

Tra i numeri abbondanti alcuni sono detti *semi-perfetti* perché sono uguali alla somma di un sottoinsieme dei loro divisori. Ad esempio il $24 = 1 + 2 + 3 + 4 + 6 + 8$ oppure $24 = 1 + 2 + 3 + 6 + 12$ o anche $24 = 4 + 8 + 12$.

Provare a verificare che un numero è semiperfetto generando tutti i sottoinsiemi dei divisori usando `<*>` sulle liste (che “moltiplica” le computazioni) e poi filtrando solo i sottoinsiemi con la somma uguale al numero (magari usando il tipo `Maybe`).

3. Nodi Equilibrati con Applicativi e Monadi

Un nodo u di un albero (considerate il tipo `BinTree` con valori numerici in tutti i nodi) è detto *equilibrato* se la somma delle chiavi nel cammino dalla radice a u (esclusa la chiave in u) è esattamente uguale alla somma delle chiavi del sotto-albero radicato in u (compresa la chiave in u).

Scrivere una funzione `nodiEquilibrati :: Num a => BinTree a -> [a]` che preso in input un albero, restituisce la lista (eventualmente vuota) contenente tutti i valori nei nodi equilibrati, facendo possibilmente un'unica scansione dell'albero.

Dare poi una soluzione che fa uso di Monadi in modo da evitare di dover usare (esplicitamente nel codice) parametri e risultati di ritorno ausiliari. Chi vuole strafare può dare anche la soluzione con Applicativi.